Transformers Intro: Two-Lecture Outline (CPSC 471)

Oct 21, 2026 · @keith

Two 75-minute lectures. The approach is intuition first, a tiny hand-worked example second, and matrices last. There is no backprop anywhere; students only need dot products and softmax, which they already know. The plan reuses existing course material that is currently commented out on the site: the BPE example, the self-attention notes, and the GloVe-vs-BERT `embeddings_demo` notebook.

Before Lecture 1: assign the deeplearning.ai "Attention in Transformers" intro video.

Lecture 1: Why attention, and how it works by hand

History: Limits of RNNs/LSTMs/GRUs for sequence tasks;

- parrallel verses sequential processing
- self-attention - they handle long range dependencies better

- RNNs Forget information in long sequences
- RNNs have a vanishing gradient problem
  - Activation function like tanh (-1<g<1) typically less than 1, say its .1
  - if 10 cycles thru; (.1)**10=0.0000000001

They are obsolete do not use them!

1. **Hook (5 min).** Use the "it was too tired / too wide" sentence pair (Worked examples). How does a model know what "it" means?
2. **Embeddings (15 min).** Meaning stored as a vector: similar words point in similar directions.
   - Plot GloVe vectors from `embeddings_demo`; show king − man + woman ≈ queen.
   - Dot product = similarity. Work the cat / dog / car example on the board (Worked examples).
   - This is the one idea attention is built on.
3. **Where the vectors come from (5 min).** Text → tokens → IDs → rows of a learned embedding table. Show Tiktokenizer, including a word that splits into subwords. Assign the BPE example as reading.
4. **The problem (10 min).** Run the GloVe "bat" cells of `embeddings_demo`. "Bat" gets the same vector in "the bat flew" and "the bat swung." Takeaway: a word's meaning depends on its neighbors.
5. **Attention as "who should I listen to?" (25 min).** Party analogy:
   - **Query:** what I'm looking for.
   - **Key:** what each word advertises about itself.

    - **Value:** what a word shares once you listen to it.
    - Score = dot product; softmax turns scores into weights (week 4); new vector = original + weighted sum of the values.
    - Work the "bat flew / bat swung" board example.
6. **Payoff and exit ticket (15 min).** Run the BERT half of the demo: "bat" now gets different vectors. Then the exit-ticket problem.

Lecture 2: From one attention step to a Transformer

1. **Recap (10 min).** Redo the board example quickly.
2. **Matrix form (15 min).** $\text{softmax}(QK^T/\sqrt{d})\cdot V$ is the same board math done for all tokens at once, so it runs in parallel on a GPU, unlike an RNN. Say √d "keeps softmax from saturating" and skip the derivation. Test 2 covers this matrix math.
3. **The block as Lego (15 min).** A transformer block = attention + an MLP (they built MLPs) + a residual connection + LayerNorm (a cousin of BatchNorm). Stack N blocks.
    - Multi-head attention is like several CNN filters: each head looks for a different kind of relationship.
    - Positional encoding exists because attention ignores word order. One slide.
4. **Three flavors (15 min).**
    - **Encoder (BERT):** sees the whole sentence; produces representations.
    - **Decoder (GPT):** causal mask, autoregressive, trains in parallel with teacher forcing.
    - **Encoder-decoder:** translation.
5. **"You've already used these" (15 min).** ViT and Swin from timm and RF-DETR from week 8 are transformers; images are cut into patches that act as tokens. Preview week 10 Hugging Face pipelines and week 11 DistilBERT fine-tuning.
6. **Wrap-up (5 min).**

Worked examples

All vectors are 2-dimensional so the arithmetic fits on a board. Read the two dimensions as [animal, sports]. Skip the √d scaling in Lecture 1.

Hook sentence pair

- "The animal didn't cross the street because **it** was too tired." → it = animal
- "The animal didn't cross the street because **it** was too wide." → it = street

One word changed, and "it" means something else. The model has to look at the neighbors.

Lecture 1 embeddings example: dot product as similarity

Word	Vector [animal, vehicle]
cat	[0.9, 0.1]
dog	[0.8, 0.2]

Word	Vector [animal, vehicle]
car	[0.1, 0.9]

- cat · dog = 0.72 + 0.02 = 0.74 → similar
- cat · car = 0.09 + 0.09 = 0.18 → not similar

Attention uses exactly this score to decide which words to listen to.

Lecture 1 tokenization example

The IDs and vectors below are made up for the board; Tiktokenizer shows real ones.

1. **Text → tokens:** "the bat flew" → [the, bat, flew]. A rarer word splits into pieces, for example "unhappiness" → [un, happiness]. Type a few words into Tiktokenizer live to show real splits.
2. **Tokens → IDs:** the = 7, bat = 42, flew = 913. The vocabulary is a fixed list (about 50,000 entries for GPT-2).
3. **IDs → vectors:** the embedding table has one row per vocabulary entry. ID 42 means "take row 42."

ID	Token	Embedding row (2-d for the board)
7	the	[0.1, 0.3]
42	bat	[0.9, 0.8]
913	flew	[1.2, 0.1]

Tie it to what they know: the lookup is a one-hot vector times the embedding matrix, so the table is just a weight matrix learned by backprop. These vectors are the starting point; attention then makes them context-aware.

Lecture 1 board example: "the bat flew"

Give the embedding, Q, K, and V directly; say "the model learns these" and save the weight matrices for Lecture 2. Read the two dimensions as [animal, sports].

Bat's original embedding = [0.9, 0.8], nearly balanced between animal and sports. Bat's query = [1, 2].

Token	Key	Value
the	[0, 1]	[0, 0]
bat	[1, 0]	[1, 1]
flew	[2, 1]	[2, 0]

1. **Scores (q·k):**
    - the: [1, 2]·[0, 1] = 1×0 + 2×1 = 2
    - bat: [1, 2]·[1, 0] = 1×1 + 2×0 = 1

    - flew: [1, 2]·[2, 1] = 1×2 + 2×1 = 4
2. **Exponentiate:** $e^2 = 7.389$, $e^1 = 2.718$, $e^4 = 54.598$. Sum = 64.705.
3. **Softmax (divide by the sum):** the = 0.114, bat = 0.042, flew = 0.844. The weights sum to 1.
4. **Weighted sum of values** (weights row × V matrix, rows the / bat / flew):
    - [0.114, 0.042, 0.844] × [[0, 0], [1, 1], [2, 0]]
    - = [0.114×0 + 0.042×1 + 0.844×2, 0.114×0 + 0.042×1 + 0.844×0]
    - = [0 + 0.042 + 1.688, 0 + 0.042 + 0] = [1.730, 0.042]
    - This is the **update**. It points toward animal; bat itself has not changed yet.
5. **Residual:** new bat = original bat + update = [0.9, 0.8] + [1.730, 0.042] = **[2.630, 0.842]**. Bat now leans animal (animal/sports ratio goes from about 1.1 to 3.1).

What's a residual? The skip connection from the ResNet paper: the block's input skips around the block and is added to its output, so output = x + Attention(x). The layer learns a change to the vector rather than a whole new vector, and gradients flow back more easily.

Now "the bat swung"

"Swung" is a different word with a different embedding, so it gets its own key and value. The weight matrices W_K and W_V do not change; they are shared by every token. "The" and "bat" keep the same keys, values, and query, because in the first layer these come only from each word's own embedding.

Token	Key	Value
the	[0, 1]	[0, 0]
bat	[1, 0]	[1, 1]
swung	[1, 1]	[0, 2]

1. **Scores:** the = 2, bat = 1, swung = [1, 2]·[1, 1] = 1×1 + 2×1 = 3
2. **Exponentiate:** $e^2 = 7.389$, $e^1 = 2.718$, $e^3 = 20.086$. Sum = 30.193.
3. **Softmax:** the = 7.389 / 30.193 = 0.245; bat = 2.718 / 30.193 = 0.090; swung = 20.086 / 30.193 = 0.665. Sum = 1.
4. **Weighted sum of values:**
    - [0.245, 0.090, 0.665] × [[0, 0], [1, 1], [0, 2]]
    - = [0 + 0.090 + 0, 0 + 0.090 + 1.330] = [0.090, 1.420]
5. **Residual:** new bat = [0.9, 0.8] + [0.090, 1.420] = **[0.990, 2.220]**. Bat now leans sports.

Same word, same starting embedding, same query, different neighbor, different final vector. This is the BERT demo done by hand. Note that the weights changed too (swung gets 0.665 instead of 0.844), because swung has a different key.

Exit ticket (answer included)

Query = [1, 0]. Keys: A = [1, 1], B = [0, 1], C = [3, 0]. Values: A = [1, 0], B = [0, 1], C = [0, 2]. Compute the output.

Answer:

1. Scores: A = [1, 0]·[1, 1] = 1; B = [1, 0]·[0, 1] = 0; C = [1, 0]·[3, 0] = 3.
2. Exponentiate: $e^1 = 2.718$, $e^0 = 1$, $e^3 = 20.086$. Sum = 23.804.
3. Weights: 2.718 / 23.804 = 0.114; 1 / 23.804 = 0.042; 20.086 / 23.804 = 0.844.
4. Output = 0.114 × [1, 0] + 0.042 × [0, 1] + 0.844 × [0, 2] = [0.114, 0] + [0, 0.042] + [0, 1.688] = [0.114, 1.730].

The weights match the board example because every score is shifted down by 1, and softmax ignores a constant shift. Worth pointing out if a student notices.

Lecture 2: the same example as matrices

Q, K, and V come from the input embeddings X times three learned weight matrices (Q = X·W_Q, and so on). Those are ordinary linear layers trained by backprop.

Stack the three queries as rows: the = [0, 1], bat = [1, 2], flew = [1, 0]. Then QK^T gives every score at once:

Query ↓ / Key →	the	bat	flew
the	1	0	1
bat	2	1	4
flew	0	1	2

Each cell is one dot product, row query · column key. For example, the "the" row: [0, 1]·[0, 1] = 1, [0, 1]·[1, 0] = 0, [0, 1]·[2, 1] = 1. The "bat" row is the board example.

Softmax each row (skipping √d for the board; dividing by √2 first would only soften the weights):

Row	Exponentials	Sum	Weights (the, bat, flew)
the	2.718, 1, 2.718	6.437	0.422, 0.155, 0.422
bat	7.389, 2.718, 54.598	64.705	0.114, 0.042, 0.844
flew	1, 2.718, 7.389	11.107	0.090, 0.245, 0.665

Multiply by V (rows: the = [0, 0], bat = [1, 1], flew = [2, 0]):

- the: 0.422 × [0, 0] + 0.155 × [1, 1] + 0.422 × [2, 0] = [1.000, 0.155]
- bat: 0.114 × [0, 0] + 0.042 × [1, 1] + 0.844 × [2, 0] = [1.730, 0.042]
- flew: 0.090 × [0, 0] + 0.245 × [1, 1] + 0.665 × [2, 0] = [1.575, 0.245]

The output is a 3×2 matrix, one new vector per token, from one matrix multiply. The "bat" row matches the board.

Did not teach below, probably do not need for 471

Lecture 2: causal mask (decoder)

Before softmax, set every future position to −∞. Since e^(−∞) = 0, no token can look ahead.

Query ↓ / Key →	the	bat	flew
the	1	−∞	−∞
bat	2	1	−∞
flew	0	1	2

Softmax each row (e^(−∞) = 0):

- the: $e^1 / e^1 = 1$ → weights [1, 0, 0]
- bat: $e^2 = 7.389$, $e^1 = 2.718$; sum = 10.107 → [0.731, 0.269, 0]
- flew: same as unmasked → [0.090, 0.245, 0.665]

Multiply by V:

- the: 1 × [0, 0] = [0, 0]
- bat: 0.731 × [0, 0] + 0.269 × [1, 1] = [0.269, 0.269]
- flew: [1.575, 0.245], unchanged, because the last token can already see everything

Each row predicts the next token, and all rows train in parallel (teacher forcing).

What to skip

Sinusoidal positional encoding formulas, KV cache, the full original paper, and training details. Leave these for week 10 or later.